

Módulos Odoo – Modelos y Vistas

Birhan Fdez

22 de abril de 2026



Índice

1. Introducción	3
2. Consideraciones previas	3
3. Actividad 01: Modificación de Lista de Tareas	5
3.1. Añadido vista Kanban	5
3.2. Nuevo campo y nueva vista Calendario	6
3.3. Comprobación de Vistas	7
4. Actividad 02: Ampliación del Módulo Biblioteca	10
4.1. Implementación de Socio	10
4.2. Implementación de Ejemplares	11
4.2.1. Restricciones de fechas	11
4.2.2. Relación con el modelo Cómic	12
4.3. Vistas y Menús	13
4.4. Comprobación de Vistas	15
5. Actividad 03: Gestión Hospitalaria	18
5.1. Paso Previo	18
5.2. Modelos de Datos y Relaciones	19
5.2.1. Modelo Paciente	19
5.2.2. Modelo Médico	19
5.2.3. Modelo Consulta	20
5.3. Vistas y Menús	20
5.4. Comprobación de Vistas	24
6. Actividad 04: Gestión de Ciclos Formativos	26
6.1. Modelos de Datos y Relaciones	27
6.1.1. Modelo Ciclo Formativo	27
6.1.2. Modelo Módulo	27
6.1.3. Modelo Alumno	28
6.1.4. Modelo Profesor	28
6.2. Vistas y Menús	28
6.3. Comprobación de Vistas	31
6.4. Configuración de seguridad	35
6.5. Comprobación de permisos	36

1. Introducción

En este documento se presenta una guía detallada sobre el desarrollo de las actividades de la **Práctica 4 del modelo de Odoo**.

El objetivo principal de esta práctica es aplicar los conocimientos adquiridos sobre los modelos relacionales y vistas a la hora de desarrollar aplicaciones en el entorno de Odoo. Para ello, implementaremos los distintos módulos que abarcan la gestión de tareas, la gestión de pacientes, médico y consultas en un hospital, la organización de ciclos formativos en un instituto y la gestión de una biblioteca de cómics.

Esta práctica pretende que el usuario comprenda la interacción de los módulos, sus relaciones y sus vistas, además de su integración con las interfaces de **Odoo** para la gestión de información.

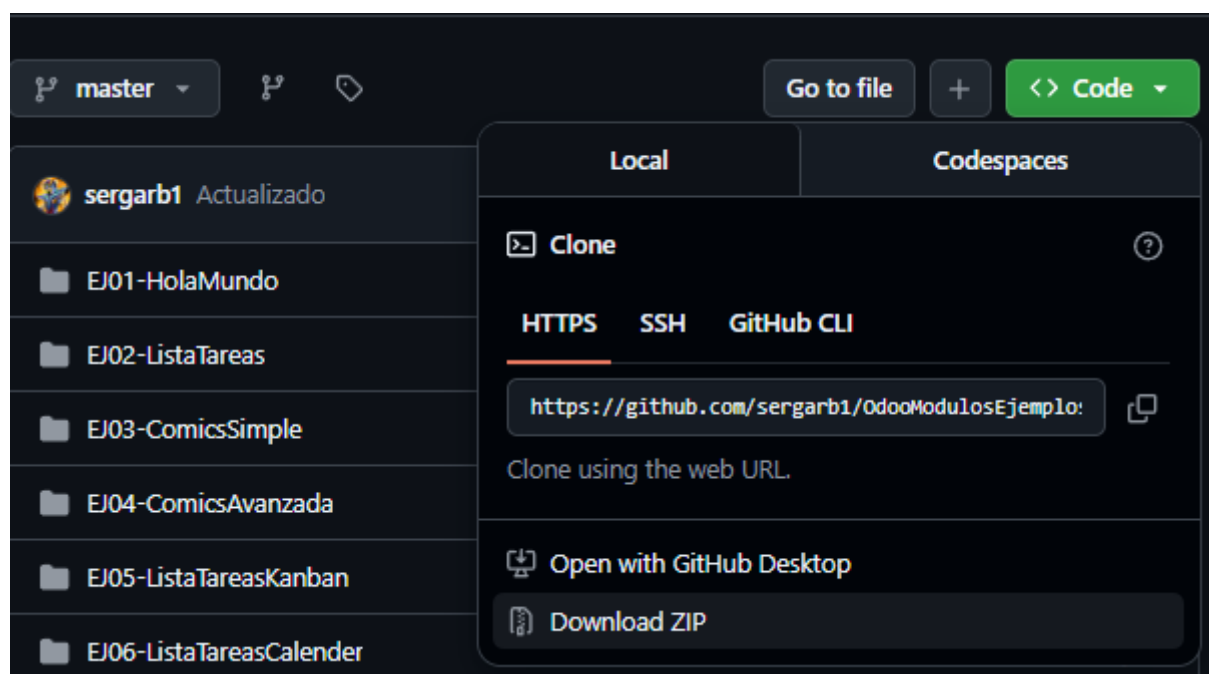
2. Consideraciones previas

Antes de comenzar con las actividades, será necesario realizar una serie de pasos previos.

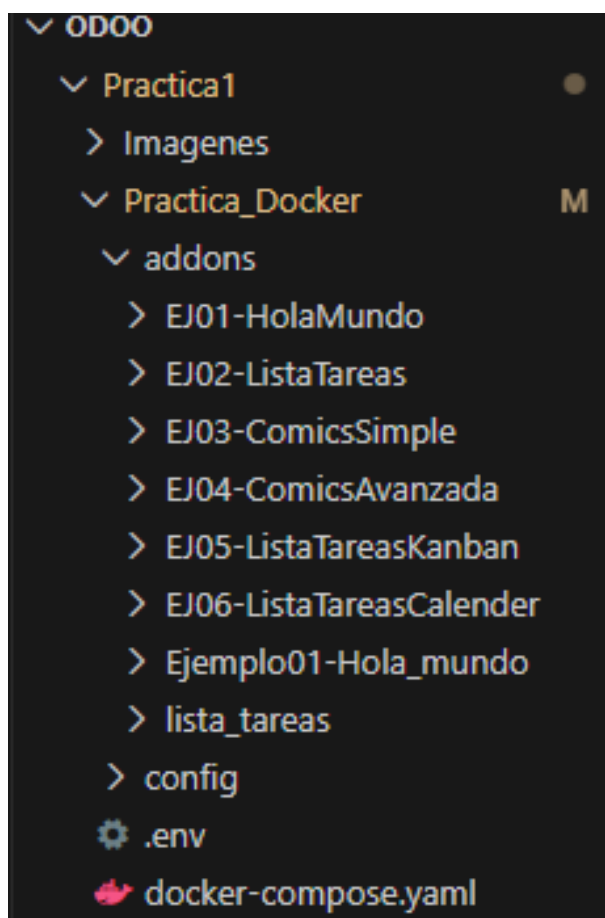
Lo primero que se debe hacer es acceder al repositorio de GitHub donde se encuentran los módulos de ejemplo proporcionados.

<https://github.com/sergarb1/OdooModulosEjemplos>

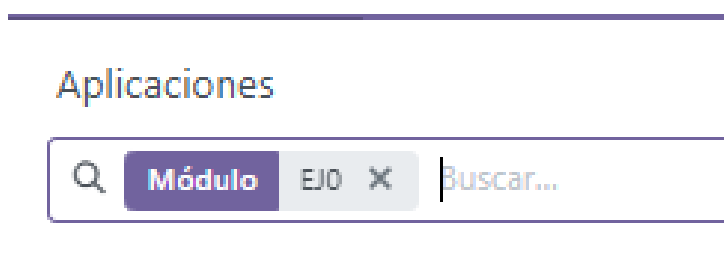
Una vez dentro del repositorio, procederemos a descargar los módulos presentes en formato **.ZIP**.

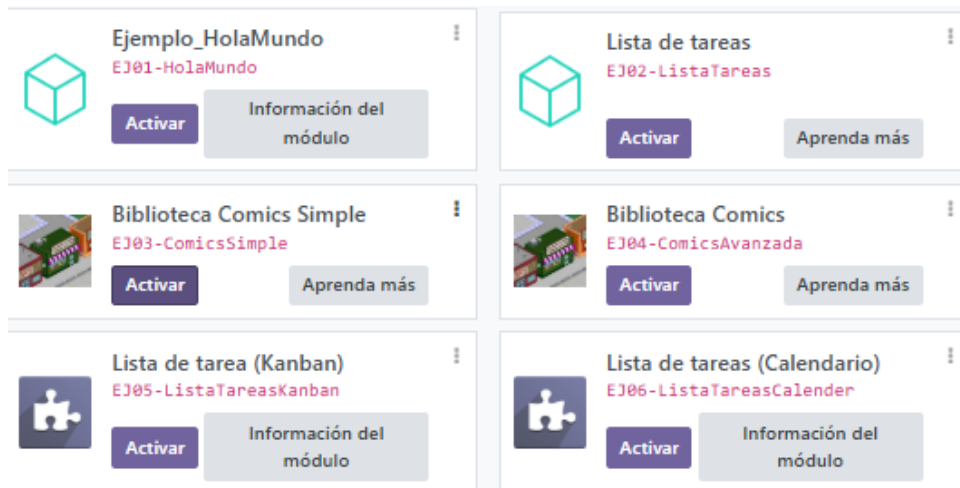


Tras finalizar la descarga, añadiremos los módulos dentro de la carpeta **Addons** del entorno de Odoo para su uso en las actividades.



Con los módulos añadidos, nos dirigiremos a Odoo para realizar una actualización de la lista de aplicaciones utilizando los permisos de administración. Posteriormente, desde el buscador de aplicaciones, introduciremos el prefijo **EJ0** para comprobar que los módulos se hayan instalado correctamente.





Una vez comprobado que los módulos han sido instalados de forma correcta, podremos dar inicio a las actividades principales de esta práctica.

3. Actividad 01: Modificación de Lista de Tareas

En esta actividad, se nos pide realizar dos modificaciones sencillas al módulo de **Lista de Tareas** desarrollado en una práctica anterior.

3.1. Añadido vista Kanban

La primera modificación que se nos pide es añadir una nueva vista en formato **Kanban**. Para ello, debemos dirigirnos al archivo **views.xml** del módulo en la carpeta **Views**.

Una vez dentro, debemos agregar el siguiente código para añadir la nueva vista a la lista de tareas.

```
<!-- Definimos como es la vista kanban-->
<record model="ir.ui.view" id="lista_tareas_kanban">
  <field name="name">lista_tareas.kanban</field>
  <field name="model">lista_tareas.lista_tareas</field>
  <field name="arch" type="xml">
    <kanban>
      <!-- Campo de título visible en la tarjeta Kanban -->
      <field name="tarea"/>
      <field name="prioridad"/>
      <field name="urgente"/>
      <templates>
        <t t-name="kanban-box">
          <div class="oe_kanban_global_click">
            <strong><field name="tarea"/></strong>
            <div>Prioridad: <field name="prioridad"/></div>
            <div t-if="record.urgente.value">Urgente</div>
          </div>
        </t>
      </templates>
    </kanban>
  </field>
</record>
```

Con el código principal de la vista añadido, nos dirigiremos al apartado de acciones y menús para activar la nueva vista. Para ello ingresamos el siguiente código:

```
<record model="ir.actions.act_window" id="lista_tareas_action_window">
  <field name="name">Listado de tareas pendientes</field>
  <field name="res_model">lista_tareas.lista_tareas</field>
  <field name="view_mode">list,form,kanban</field>
</record>

<!-- Top menu item -->
<menuitem name="Listado de tareas" id="lista_tareas_menu_root"/>
<!-- menu categories -->
<menuitem name="Opciones Lista Tareas" id="lista_tareas_menu_1" parent="lista_tareas_menu_root"/>
<!-- actions -->
<menuitem name="Mostrar lista" id="lista_tareas.menu_1_list" parent="lista_tareas_menu_1" action="lista_tareas_action_window"/>
```

3.2. Nuevo campo y nueva vista Calendario

Con la vista **Kanban** añadida, el segundo apartado de la actividad nos pide modificar los campos de las tareas añadiendo el nuevo campo **fecha asignada**. Esto se realiza desde el archivo **models.py** en la carpeta **Models**. Dentro, añadiremos el siguiente campo:

Nuevo Campo Fecha

```
fecha_vencimiento = fields.Date(string="Fecha de Vencimiento")
```

Una vez agregado este nuevo campo, la clase lista quedará de esta forma:

```
#Definimos el modelo de datos
class ListaTareas(models.Model):
    #Nombre y descripcion del modelo de datos
    _name = 'lista_tareas.lista_tareas'
    _description = 'lista_tareas.lista_tareas'

    #Elementos de cada fila del modelo de datos
    #Los tipos de datos a usar en el ORM son
    #https://www.odoo.com/documentation/17.0/developer/reference/addons/orm.html
    #fields
    tarea = fields.Char(string="Tarea")
    prioridad = fields.Integer(string="Prioridad")
    urgente = fields.Boolean(compute='_value_urgente', store=True)
    realizada = fields.Boolean(string="Realizada")
    fecha_vencimiento = fields.Date(string="Fecha de Vencimiento")
    #Este computo depende de la variable prioridad
    @api.depends('prioridad')
    #Funcion para calcular el valor de urgente
    def _value_urgente(self):
        for record in self:
            #Si la prioridad es mayor que 10, se considera urgente, en otro caso no lo es
            if record.prioridad>10:
                record.urgente = True
            else:
                record.urgente = False
```

Tras completar la modificación de la clase, nos volveremos a dirigir al **apartado de vistas**, donde nos pide añadir una vista de calendario así como actualizar el apartado de acciones.

Para ello, ingresaremos el siguiente código que gestionará la vista calendario.

Nueva Vista Calendario

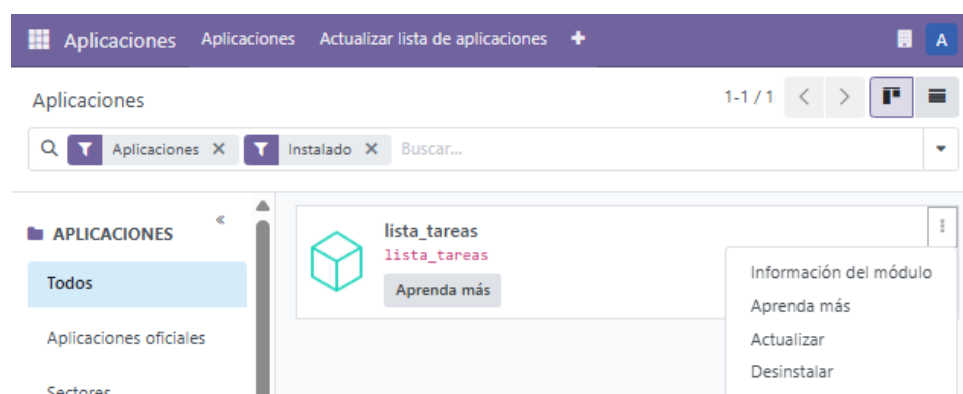
```
<!-- definimos como es la vista calendario-->
<record id="lista_tareas_calendar_view" model="ir.ui.view">
  <field name="name">lista_tareas.calendar</field>
  <field name="model">lista_tareas.lista_tareas</field>
  <field name="arch" type="xml">
    <calendar date_start="fecha_vencimiento" string="Tareas por fecha">
      <field name="tarea"/>
      <field name="prioridad"/>
    </calendar>
  </field>
</record>
```

```
<record model="ir.actions.act_window" id="lista_tareas_action_window">
  <field name="name">Listado de tareas pendientes</field>
  <field name="res_model">lista_tareas.lista_tareas</field>
  <field name="view_mode">list,form,kanban,calendar</field>
</record>
```

3.3. Comprobación de Vistas

Con las nuevas modificaciones implementadas en el módulo, realizaremos una comprobación actualizando el módulo y haciendo una prueba.

Para ello, empleando los filtros, localizaremos nuestro módulo **Lista_Tareas**. Una vez localizado, lo actualizamos para implementar los nuevos cambios.



Volver al índice

Una vez completada la actualización, ingresaremos al módulo para realizar una prueba cuyo objetivo es confirmar que las modificaciones funcionan correctamente.

☐ Tarea	Prioridad Urgente	Realizada	Fecha de Vencimiento
--------------------------------	-------------------	-----------	----------------------

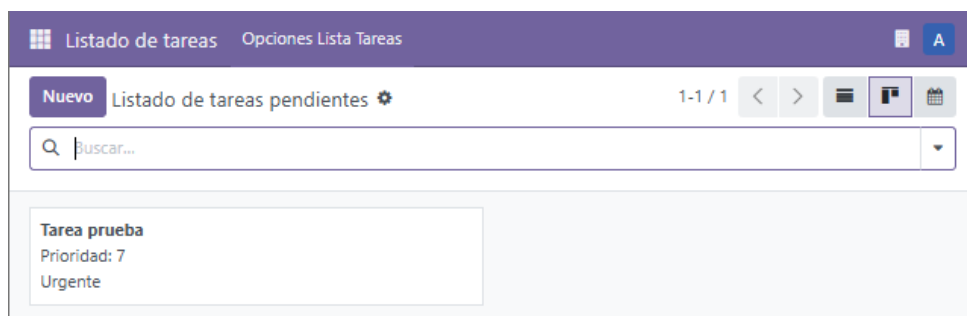
Para ello, crearemos una nueva tarea que incluirá el nuevo campo **Fecha de Vencimiento**.

Tarea	Tarea prueba
Urgente	☐
Fecha de Vencimiento	12/12/2025
Prioridad	7
Realizada	☐

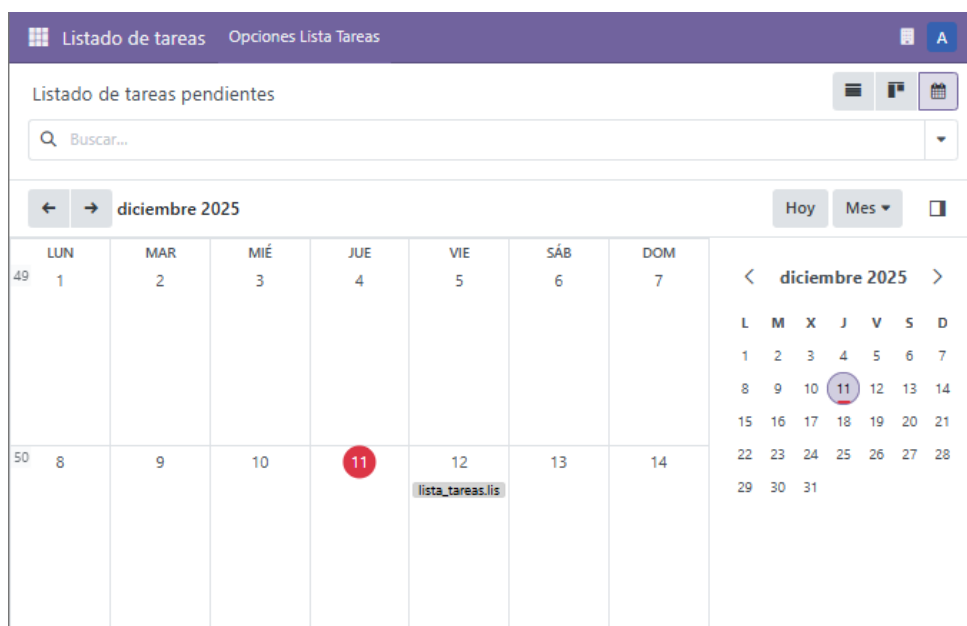
☐ Tarea	^	Prioridad	Urgente	Realizada	Fecha de ...
☐ Tarea prueba		7	☐	☐	12/12/2025

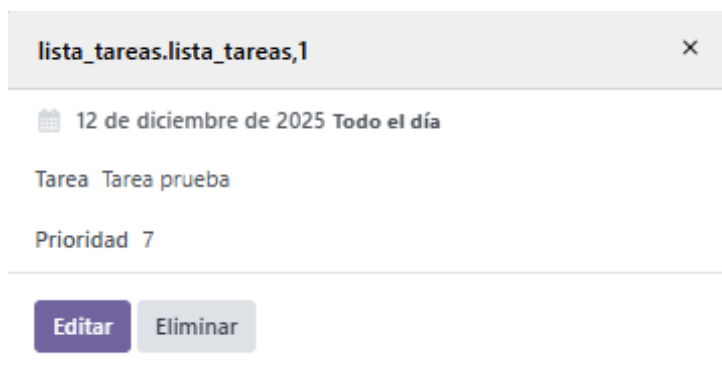
Con la nueva tarea de prueba creada, nos dispondremos a navegar por las nuevas vistas observando que no tengamos ningún problema y que están funcionales.

La primera vista que comprobaremos es la vista **Kanban**.



Seguido de la vista **Calendario**, que nos mostrará en la fecha asignada la nueva tarea.





The screenshot shows a modal window titled 'lista_tareas.lista_tareas,1' with a close button. Inside, it displays a calendar icon followed by '12 de diciembre de 2025 Todo el día'. Below this, the task is listed as 'Tarea Tarea prueba' and its priority is 'Prioridad 7'. At the bottom, there are two buttons: 'Editar' (highlighted in purple) and 'Eliminar'.

Con la comprobación y confirmación de que todos los nuevos cambios realizados en el módulo **Lista_Tareas** funcionan perfectamente, daremos por completada esta actividad.

4. Actividad 02: Ampliación del Módulo Biblioteca

En esta actividad, se nos indica realizar una ampliación del módulo añadiendo dos nuevas clases, así como sus restricciones correspondientes.

4.1. Implementación de Socio

La primera implementación que se nos pide es **crear la nueva clase Socio**, que almacenará los datos (**Nombre, Apellidos e Identificador**).

Para realizar esta implementación, desde el archivo **biblioteca_comic.py** en **Models** crearemos la siguiente clase:

```
Nueva Clase Socio

#Definimos la clase socio
class Socio(models.Model):
    #Nombre y descripcion del modelo
    _name = 'biblioteca.socio'
    _description = 'Socio de la biblioteca'
    _rec_name = 'identificador'

    #Atributos
    nombre = fields.Char('Nombre', required=True)
    apellido = fields.Char('Apellido', required=True)
    identificador = fields.Char('Identificador', required=True, help='ID único del socio')
```

4.2. Implementación de Ejemplares

Con la **clase Socio** implementada, a continuación crearemos la **clase Ejemplar** para gestionar los préstamos. Esta clase será la encargada de representar cada copia física de un cómic disponible para préstamo, relacionando cada ejemplar con el cómic original y con el socio que lo tenga en posesión en caso de estar prestado.

Para ello, añadiremos la siguiente clase en el archivo **biblioteca_comic.py** dentro del módulo **Models**:

Nueva Clase Ejemplar

```
#Definimos la clase Ejemplares
class Ejemplar(models.Model):
    #Nombre y descripción del modelo
    _name = 'biblioteca.ejemplar'
    _description = 'Ejemplar de comic para préstamo'
    _rec_name = 'comic_id'

    #Atributos
    comic_id = fields.Many2one('biblioteca.comic', string='Comic', required=True, ondelete='cascade')
    prestado_a = fields.Many2one('biblioteca.socio', string='Prestado a')
    fecha_prestamo = fields.Date('Fecha de préstamo')
    fecha_devolucion = fields.Date('Fecha prevista de devolución')

    estado = fields.Selection(
        [('disponible', 'Disponible'),
         ('prestado', 'Prestado')],
        string='Estado',
        compute='_compute_estado',
        store=True
    )
```

4.2.1. Restricciones de fechas

Una vez definida la clase, añadiremos unas **restricciones encargadas de controlar** que la fecha de préstamo no sea posterior al día actual, y que la fecha de devolución no sea anterior al día actual.

Para ello, añadimos el siguiente método:

Restricciones del modelo Ejemplar

```
@api.depends('prestado_a', 'fecha_prestamo', 'fecha_devolucion')
def _compute_estado(self):
    for rec in self:
        if rec.prestado_a and rec.fecha_prestamo and rec.fecha_devolucion:
            rec.estado = 'prestado'
        else:
            rec.estado = 'disponible'

@api.constrains('fecha_prestamo')
def _check_fecha_prestamo(self):
    today = fields.Date.today()
    for record in self:
        if record.fecha_prestamo and record.fecha_prestamo > today:
            raise ValidationError('La fecha de préstamo no puede ser posterior a hoy.')

@api.constrains('fecha_devolucion')
def _check_fecha_devolucion(self):
    today = fields.Date.today()
    for record in self:
        if record.fecha_devolucion and record.fecha_devolucion < today:
            raise ValidationError('La fecha de devolución no puede ser anterior a hoy.')
```

4.2.2. Relación con el modelo Cómic

Para finalizar con las restricciones, el modelo de cómic deberá estar relacionado con los ejemplares.

Relación con Cómic

```
@api.onchange('prestado_a')
def _onchange_prestado_a(self):
    if self.prestado_a:
        self.estado = 'prestado'
    else:
        self.estado = 'disponible'
```

Con los nuevos códigos implementados, el archivo **biblioteca_comic.py** quedará completo, lo que nos permitirá implementar los nuevos cambios necesarios en la sección de vistas.

4.3. Vistas y Menús

Finalizada la implementación de las nuevas clases y sus restricciones correspondientes, nos dirigiremos al archivo **biblioteca_comic.xml** en **Views** para definir las vistas, así como sus menús encargados de gestionar la información de cada una de forma clara.

Nueva Vista de Socio

```
<!-- ===== -->
<!-- VISTAS SOCIOS -->
<!-- ===== -->
<record id="view_socio_list" model="ir.ui.view">
  <field name="name">Lista Socios</field>
  <field name="model">biblioteca.socio</field>
  <field name="arch" type="xml">
    <list>
      <field name="identificador"/>
      <field name="nombre"/>
      <field name="apellido"/>
    </list>
  </field>
</record>

<record id="view_socio_form" model="ir.ui.view">
  <field name="name">Formulario Socio</field>
  <field name="model">biblioteca.socio</field>
  <field name="arch" type="xml">
    <form>
      <group>
        <field name="identificador"/>
        <field name="nombre"/>
        <field name="apellido"/>
      </group>
    </form>
  </field>
</record>
```

Nuevo Menú de Socio

```
<!-- =====
ACCIONES Y MENÚS SOCIOS
===== -->
<record id="action_socio" model="ir.actions.act_window">
  <field name="name">Socios</field>
  <field name="res_model">biblioteca.socio</field>
  <field name="view_mode">list,form</field>
</record>

<menuitem id="menu_socio" name="Socios" parent="biblioteca_base_menu" action="action_socio"/>
```

Nueva Vista de Ejemplares

```
<!-- =====
VISTAS EJEMPLARES
===== -->
<record id="view_ejemplar_list" model="ir.ui.view">
  <field name="name">Lista Ejemplares</field>
  <field name="model">biblioteca.ejemplar</field>
  <field name="arch" type="xml">
    <list>
      <field name="comic_id"/>
      <field name="prestado_a"/>
      <field name="fecha_prestamo"/>
      <field name="fecha_devolucion"/>
      <field name="estado"/>
    </list>
  </field>
</record>

<record id="view_ejemplar_form" model="ir.ui.view">
  <field name="name">Formulario Ejemplar</field>
  <field name="model">biblioteca.ejemplar</field>
  <field name="arch" type="xml">
    <form>
      <group>
        <field name="comic_id"/>
        <field name="prestado_a"/>
        <field name="fecha_prestamo"/>
        <field name="fecha_devolucion"/>
        <field name="estado" readonly="1"/>
      </group>
    </form>
  </field>
</record>
```

Nuevo Menú de Ejemplares

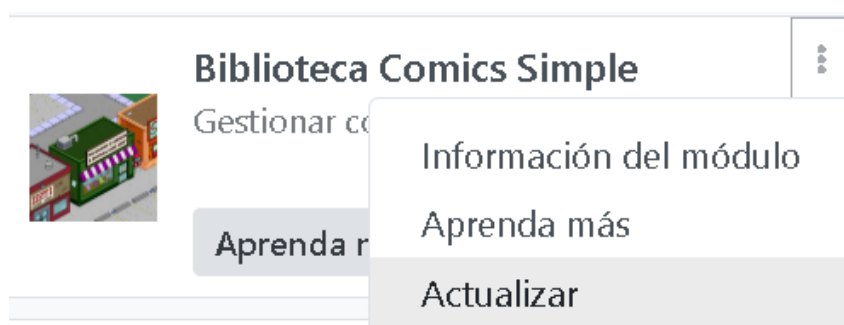
```
<!-- ===== -->
<!-- ACCIONES Y MENÚS EJEMPLARES -->
<!-- ===== -->
<record id="action_ejemplar" model="ir.actions.act_window">
  <field name="name">Ejemplares</field>
  <field name="res_model">biblioteca.ejemplar</field>
  <field name="view_mode">list,form</field>
</record>

<menuitem id="menu_ejemplar" name="Ejemplares" parent="biblioteca_base_menu" action="action_ejemplar"/>
```

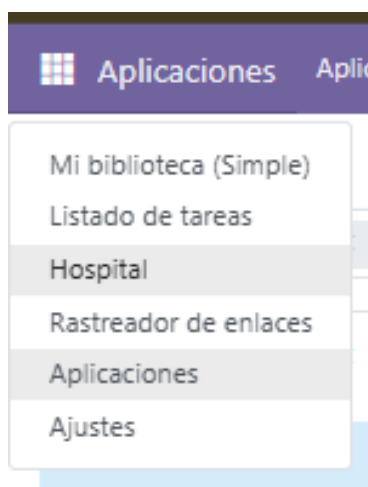
Finalizada la implementación de las nuevas vistas y menús de forma correcta, realizaremos una comprobación desde Odoo para dar por finalizadas las nuevas modificaciones que se nos indican.

4.4. Comprobación de Vistas

Para realizar la comprobación, desde la sección de módulos instalados de nuestro Odoo, actualizaremos el módulo **Biblioteca Comics Simple**.



Una vez realizada la actualización, ingresaremos al módulo desde el menú de aplicaciones.



Lo primero que observaremos, en caso de que se haya realizado todo correctamente, son las vistas de los menús de navegación entre cómics, ejemplares y socios.

 Mi biblioteca (Simple) Comics Socios Ejemplares

Nuevo

Biblioteca de Comics 

 Buscar...

Dentro de la vista de **cómics**, podremos observar que se nos muestran los datos de forma correcta.

 Mi biblioteca (Simple) Comics Socios Ejemplares

Nuevo

Biblioteca de Comics 

 Buscar...

Así como en la vista de los **Socios**.

 Mi biblioteca (Simple) Comics Socios Ejemplares

Nuevo

Socios 

 Buscar...

Al crear un nuevo socio, podremos observar cómo, en caso de que no se ingrese todos los datos necesarios, el usuario recibirá una alerta de error.

The screenshot shows a web form titled 'Nuevo Socio'. The form has fields for 'Identificador', 'Nombre', and 'Apellido'. The 'Identificador' field is highlighted in red, indicating an error. A modal dialog box is open, displaying the message 'Campos no válidos:' followed by a list containing 'Identificador'. The 'Nombre' field contains the text 'Birhan' and the 'Apellido' field contains 'Fdez Fdez'.

Seguido de la vista de **los ejemplares**.

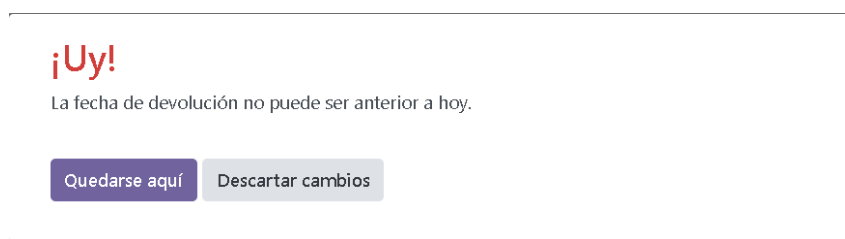
The screenshot shows the 'Ejemplares' (Copies) view. At the top, there is a navigation bar with 'Mi biblioteca (Simple)', 'Comics', 'Socios', and 'Ejemplares'. Below this is a search bar with the placeholder 'Buscar...'. The main content is a table with the following columns: 'Comic', 'Prestado a', 'Fecha de ...', 'Fecha pre...', and 'Estado'. The table contains two rows of data:

Comic	Prestado a	Fecha de ...	Fecha pre...	Estado
CAPITÁN AMÉRICA 14	IAG	24/05/2026	29/05/2026	Prestado
EL ASOMBROSO SPIDERMAN	RPG	24/05/2026	01/06/2026	Prestado

Donde podremos observar que, en caso de que el usuario introduzca una fecha de préstamo posterior a la fecha actual, este recibirá una alerta de error.

The screenshot shows an error alert dialog box. It has a red exclamation mark icon and the text '¡Uy!' in red. Below this, it says 'La fecha de préstamo no puede ser posterior a hoy.' At the bottom, there are two buttons: 'Quedarse aquí' (purple) and 'Descartar cambios' (gray).

Así como en caso de que la devolución sea en una fecha anterior a la del registro del préstamo.



Con esto, podemos observar que las modificaciones que se nos piden en la actividad se cumplen y ejecutan de forma correcta, pudiendo así finalizar esta actividad.

5. Actividad 03: Gestión Hospitalaria

En esta actividad se nos indica crear un módulo de Odoo cuyas funciones son la gestión básica de un hospital. Este módulo tendrá que permitir la administración de pacientes, médicos y consultas.

5.1. Paso Previo

Para comenzar, lo primero que realizaremos es crear el módulo **Hospital** desde el Docker donde realizamos esta actividad. Esto lo podemos realizar empleando el siguiente código:

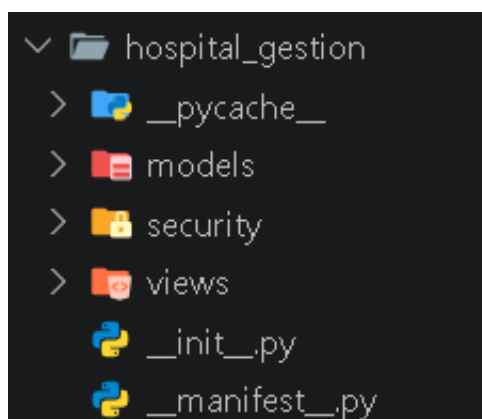
Comando Para Ingresar al Docker

```
docker exec -it -u root e0.bigodoo.net /bin/bash
```

Una vez dentro de nuestro Docker, emplearemos este comando:

Comando Para Crear Módulo Básico

```
odoo scaffold gestion_hospital /mnt/hospital_gestion
```



Con el módulo básico creado, nos pondremos a trabajar en los elementos que se nos piden en la actividad.

5.2. Modelos de Datos y Relaciones

5.2.1. Modelo Paciente

Comenzaremos con el modelo del paciente. Para crear la clase de paciente, nos dirigimos al archivo **models.py** en la **sección de models**. Una vez dentro, crearemos la siguiente clase gestora con los atributos que se nos indican en la actividad.

Código Clase Paciente

```
from odoo import models, fields, api
from odoo.exceptions import ValidationError

#Definimos modelo Paciente
class Paciente(models.Model):
    #Nombre y descripcion del modelo
    _name='hospital.paciente'
    _description='Paciente del hospital'
    # Atributo que se mostrará por defecto como nombre del registro
    _rec_name='nombre'

    #Atributos
    nombre = fields.Char('Nombre', required=True)
    apellidos = fields.Char('Apellidos', required=True)
    sintomas = fields.Text('Sintomas')
```

5.2.2. Modelo Médico

Con la clase del paciente completada, lo siguiente que realizaremos es la creación de la **clase Médico**. Esta clase contendrá los atributos **Nombre**, **Apellidos** y **Número de colegiado**.

Código Clase Médico

```
#Definimos modelo Médico
class Medico(models.Model):
    #Nombre y descripcion del modelo
    _name='hospital.medico'
    _description='Médico del hospital'
    _rec_name='nombre'

    #Atributos
    nombre = fields.Char('Nombre', required=True)
    apellidos = fields.Char('Apellidos', required=True)
    # Id único del médico
    numero_colegiado=fields.Char('Número de colegiado', required=True, help='Identificador único del médico')
```

5.2.3. Modelo Consulta

Una vez finalizada la creación del módulo de médico, nos dispondremos a crear el módulo de **Consulta**. Antes de continuar, se debe tener en cuenta que este es el módulo principal, ya que en él se indican las relaciones entre las clases de médico y paciente. Una vez destacado esto, deberemos implementar el siguiente código.

Código Clase Consulta

```
#Definimos modelo Consulta
class Consulta(models.Model):
    _name='hospital.consulta'
    _description='Consulta entre paciente y médico'
    _rec_name='paciente_id'

    # Relación Many2One con Paciente
    paciente_id = fields.Many2One('hospital.paciente',string='Paciente',required=True)
    # Relación Many2One con Médico
    medico_id = fields.Many2One('hospital.medico',string='Médico',required=True)
    # Diagnóstico de la consulta
    diagnostico = fields.Text('Diagnóstico')
    # Fecha de la consulta
    fecha_consulta = fields.Date('Fecha de la consulta',default=fields.Date.today)

    # Validación para que la fecha no sea anterior a el día de consulta
    @api.constrains('fecha_consulta')
    def _check_fecha_consulta(self):
        for record in self:
            if record.fecha_consulta and record.fecha_consulta > fields.Date.today():
                raise ValidationError('La fecha de la consulta no puede ser posterior a hoy.')
```

Tras completar el código de la clase **consulta**, nos dirigiremos a la sección de vistas para completar la actividad.

5.3. Vistas y Menús

En este apartado de la actividad, nos dispondremos a codificar el formato de las vistas, así como el menú de navegación que permitirá al usuario moverse entre cada una de las gestiones.

Para comenzar, en la carpeta **Views** crearemos el archivo **hospital_views.xml**. Con el archivo creado, implementaremos las vistas y los menús que permitirán a los usuarios interactuar con el módulo.

Código Vista Paciente

```
<!-- Vista Pacientes-->
<record id="view_paciente_list" model="ir.ui.view">
  <field name="name">hospital.paciente.list</field>
  <field name="model">hospital.paciente</field>
  <field name="arch" type="xml">
    <list>
      <field name="nombre"/>
      <field name="apellidos"/>
      <field name="sintomas"/>
    </list>
  </field>
</record>

<record id="view_paciente_form" model="ir.ui.view">
  <field name="name">hospital.paciente.form</field>
  <field name="model">hospital.paciente</field>
  <field name="arch" type="xml">
    <form>
      <group>
        <field name="nombre"/>
        <field name="apellidos"/>
      </group>
      <group>
        <field name="sintomas"/>
      </group>
    </form>
  </field>
</record>
```

Código Vista Médico

```
<!-- Vista Médicos-->
<record id="view_medico_list" model="ir.ui.view">
  <field name="name">hospital.medico.list</field>
  <field name="model">hospital.medico</field>
  <field name="arch" type="xml">
    <list>
      <field name="nombre"/>
      <field name="apellidos"/>
      <field name="numero_colegiado"/>
    </list>
  </field>
</record>

<record id="view_medico_form" model="ir.ui.view">
  <field name="name">hospital.medico.form</field>
  <field name="model">hospital.medico</field>
  <field name="arch" type="xml">
    <form>
      <group>
        <field name="nombre"/>
        <field name="apellidos"/>
        <field name="numero_colegiado"/>
      </group>
    </form>
  </field>
</record>
```

Código Vista Consulta

```
<!-- Vista Consultas-->
<record id="view_consulta_list" model="ir.ui.view">
  <field name="name">hospital.consulta.list</field>
  <field name="model">hospital.consulta</field>
  <field name="arch" type="xml">
    <list>
      <field name="fecha_consulta"/>
      <field name="paciente_id"/>
      <field name="medico_id"/>
      <field name="diagnostico"/>
    </list>
  </field>
</record>

<record id="view_consulta_form" model="ir.ui.view">
  <field name="name">hospital.consulta.form</field>
  <field name="model">hospital.consulta</field>
  <field name="arch" type="xml">
    <form>
      <group>
        <field name="fecha_consulta"/>
        <field name="paciente_id"/>
        <field name="medico_id"/>
      </group>
      <group>
        <field name="diagnostico"/>
      </group>
    </form>
  </field>
</record>
```

Código Menús de Navegación

```
<!-- Acciones del modulo-->
<record id="action_pacientes" model="ir.actions.act_window">
  <field name="name">Pacientes</field>
  <field name="res_model">hospital.paciente</field>
  <field name="view_mode">list,form</field>
</record>

<record id="action_medicos" model="ir.actions.act_window">
  <field name="name">Médicos</field>
  <field name="res_model">hospital.medico</field>
  <field name="view_mode">list,form</field>
</record>

<record id="action_consultas" model="ir.actions.act_window">
  <field name="name">Consultas</field>
  <field name="res_model">hospital.consulta</field>
  <field name="view_mode">list,form</field>
</record>

<!-- Menus-->
<menuitem id="menu_hospital_root" name="Hospital"/>
<menuitem id="menu_hospital_pacientes" name="Pacientes" parent="menu_hospital_root" action="action_pacientes"/>
<menuitem id="menu_hospital_medicos" name="Médicos" parent="menu_hospital_root" action="action_medicos"/>
<menuitem id="menu_hospital_consultas" name="Consultas" parent="menu_hospital_root" action="action_consultas"/>
```

Con las vistas y los menús correspondientes implementados de forma correcta, a continuación realizaremos una prueba de comprobación para confirmar el correcto funcionamiento del módulo.

5.4. Comprobación de Vistas

Para iniciar esta comprobación, debemos reiniciar nuestro Docker para que Odoo pueda aplicar los nuevos cambios y añadir el nuevo módulo.

Una vez completado el reinicio, empleando la barra de búsqueda ingresaremos el nombre de nuestro módulo para poder instalarlo y comenzar a interactuar con él.



Volver al índice

Con el módulo operativo, ingresaremos y realizaremos una prueba creando un paciente, un médico y una consulta.

Hospital Pacientes Médicos Consultas

Nuevo Pacientes

1-1 / 1

Q

Buscar...

☐	Nombre	Apellidos	Sintomas
☐	Birhan	Fdez	Dolor muscular en hombre

Hospital Pacientes Médicos Consultas

Nuevo Médicos

1-1 / 1

Q

Buscar...

☐	Nombre	Apellidos	Número de colegiado
☐	Eder	Ivan	12345

Hospital Pacientes Médicos Consultas

Nuevo Consultas

1-1 / 1

Q

Buscar...

☐	Fecha de ...	Paciente	Médico	Diagnóstico
☐	15/12/2025	Birhan	Eder	Entrenamiento fisiológico durante dos semanas

Si la implementación del módulo se realizó de forma correcta y no se detectó ningún fallo, podremos dar por concluida esta actividad.

6. Actividad 04: Gestión de Ciclos Formativos

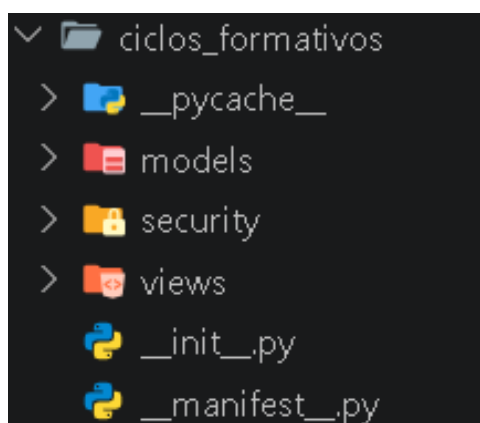
En esta actividad, se nos indica crear un módulo cuya función es **la gestión de ciclos formativos, módulos, alumnos y profesores** en un instituto. Para ello, emplearemos el mismo método que se usó en el módulo de la actividad anterior.

Comandos Para Ingresar al Docker y Crear el Módulo Básico

```
# Comando Ingreso a Docker
docker exec -it -u root e0.bigodoo.net /bin/bash

# Comando Creación Módulo Básico
odoo scaffold ciclos_formativos /mnt/ciclos_formativos
```

Una vez ejecutados estos comandos, el usuario observará la aparición de una nueva carpeta llamada **ciclos_formativos**, donde realizará la creación e implementación de los modelos, así como de sus vistas y la configuración de los permisos.



6.1. Modelos de Datos y Relaciones

6.1.1. Modelo Ciclo Formativo

En esta sección nos disponemos a desarrollar el modelo de ciclo formativo, donde implementaremos sus atributos, así como su nombre y descripción.

Código Modelo Ciclos Formativos

```
#Definimos modelo Ciclo Formativo
class cicloformativos(models.Model):
    #Nombre y descripcion del modelo
    _name='ciclo.formativo'
    _description='Ciclo Formativo'
    _rec_name='nombre'

    #Atributos
    nombre = fields.Char('Nombre', required=True)
    modulo_ids=fields.One2many('modulo', 'ciclo_id', string='Módulos')
```

6.1.2. Modelo Módulo

A continuación se muestra el código de la clase **Módulo**, que gestionará los módulos con los que contará el ciclo. En él se puede observar que este módulo está relacionado con los ciclos, los profesores y los alumnos.

Código Modelo Módulo

```
#Definimos modelo Módulo
class Modulo(models.Model):
    #Nombre y descripcion del modelo
    _name='modulo'
    _description='Módulo del ciclo formativo'
    _rec_name='nombre'

    #Atributos
    nombre=fields.Char('Nombre del módulo',required=True)
    ciclo_id=fields.Many2one('ciclo.formativo',string='Ciclo formativo', required=True)
    profesor_id=fields.Many2one('ciclo.profesor',string='Profesor que imparte')
    alumno_ids=fields.Many2many('ciclo.alumno',string='Alumnos matriculados')
```

6.1.3. Modelo Alumno

En el código que se muestra a continuación, se muestra la implementación del modelo de alumnos con sus atributos correspondientes, así como sus relaciones.

Código Modelo Alumno

```
#Definimos modelo Alumno
class Alumno(models.Model):
    #Nombre y descripción del modelo
    _name='ciclo.alumno'
    _description='Alumno del instituto'
    _rec_name='nombre'

    #Atributos
    nombre=fields.CharField('Nombre',required=True)
    modulo_ids=fields.ManyToManyField('modulo',string='Módulos inscritos')
```

6.1.4. Modelo Profesor

En este apartado implementamos la clase **Profesor**.

Código Modelo Profesor

```
#Definimos modelo Profesor
class Profesor(models.Model):
    #Nombre y descripción del modelo
    _name='ciclo.profesor'
    _description='Profesor del instituto'
    _rec_name='nombre'

    #Atributos
    nombre=fields.CharField('Nombre',required=True)
    modulo_ids = fields.OneToOneField('modulo', 'profesor_id', string='Módulos que imparte')
```

Con todo lo anterior finalizado, nos podremos dirigir a la sección de vistas, donde codificaremos las vistas del módulo, así como su menú.

6.2. Vistas y Menús

Con los modelos implementados, a continuación implementaremos las vistas con las que interactuará el usuario.

Para comenzar, lo primero que implementaremos será la vista de ciclos.

Código Vista Ciclos

```
<!-- Vistas Ciclos -->
<record id="view_ciclo_formativo_list" model="ir.ui.view">
  <field name="name">ciclo.formativo.list</field>
  <field name="model">ciclo.formativo</field>
  <field name="arch" type="xml">
    <list>
      <field name="nombre"/>
    </list>
  </field>
</record>

<record id="view_ciclo_formativo_form" model="ir.ui.view">
  <field name="name">ciclo.formativo.form</field>
  <field name="model">ciclo.formativo</field>
  <field name="arch" type="xml">
    <form>
      <field name="nombre"/>
      <field name="modulo_ids"/>
    </form>
  </field>
</record>
```

Seguido de la vista de módulo.

Código Vista Módulo

```
<!-- Vistas Módulo -->
<record id="view_modulo_list" model="ir.ui.view">
  <field name="name">modulo.list</field>
  <field name="model">modulo</field>
  <field name="arch" type="xml">
    <list>
      <field name="nombre"/>
      <field name="ciclo_id"/>
      <field name="profesor_id"/>
    </list>
  </field>
</record>

<record id="view_modulo_form" model="ir.ui.view">
  <field name="name">modulo.form</field>
  <field name="model">modulo</field>
  <field name="arch" type="xml">
    <form>
      <field name="nombre"/>
      <field name="ciclo_id"/>
      <field name="profesor_id"/>
      <field name="alumno_ids"/>
    </form>
  </field>
</record>
```

La vista de Alumno.

Código Vista Alumno

```
<!-- Vistas Alumno -->
<record id="view_alumno_list" model="ir.ui.view">
  <field name="name">alumno.list</field>
  <field name="model">ciclo.alumno</field>
  <field name="arch" type="xml">
    <list>
      <field name="nombre"/>
    </list>
  </field>
</record>

<record id="view_alumno_form" model="ir.ui.view">
  <field name="name">alumno.form</field>
  <field name="model">ciclo.alumno</field>
  <field name="arch" type="xml">
    <form>
      <field name="nombre"/>
      <field name="modulo_ids"/>
    </form>
  </field>
</record>
```

Y por último, la vista de Profesor.

Código Vista Profesor

```
<!-- Vistas Profesor -->
<record id="view_profesor_list" model="ir.ui.view">
  <field name="name">profesor.list</field>
  <field name="model">ciclo.profesor</field>
  <field name="arch" type="xml">
    <list>
      <field name="nombre"/>
    </list>
  </field>
</record>

<record id="view_profesor_form" model="ir.ui.view">
  <field name="name">profesor.form</field>
  <field name="model">ciclo.profesor</field>
  <field name="arch" type="xml">
    <form>
      <field name="nombre"/>
      <field name="modulo_ids"/>
    </form>
  </field>
</record>
```

Con las vistas finalizadas, a continuación implementaremos los menús y las acciones correspondientes para poder navegar entre cada uno de los modelos.

Código Acciones y Menús

```
<!-- Acciones y Menús -->
<record id="action_ciclo_formativo" model="ir.actions.act_window">
  <field name="name">Ciclos Formativos</field>
  <field name="res_model">ciclo.formativo</field>
  <field name="view_mode">list,form</field>
</record>

<record id="action_modulo" model="ir.actions.act_window">
  <field name="name">Módulos</field>
  <field name="res_model">modulo</field>
  <field name="view_mode">list,form</field>
</record>

<record id="action_alumno" model="ir.actions.act_window">
  <field name="name">Alumnos</field>
  <field name="res_model">ciclo.alumno</field>
  <field name="view_mode">list,form</field>
</record>

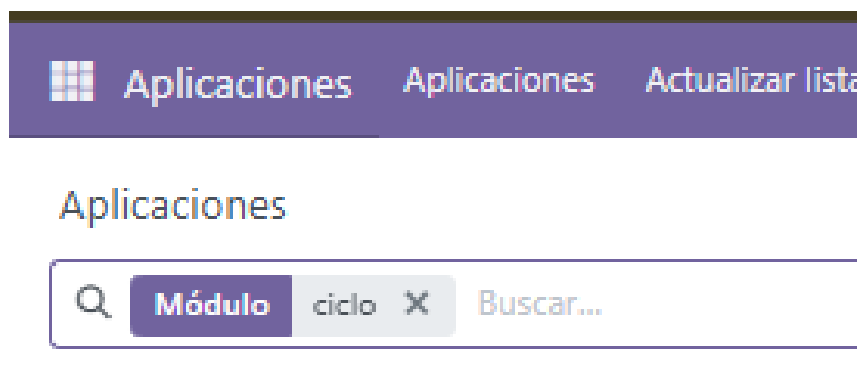
<record id="action_profesor" model="ir.actions.act_window">
  <field name="name">Profesores</field>
  <field name="res_model">ciclo.profesor</field>
  <field name="view_mode">list,form</field>
</record>

<!-- Menús -->
<menuitem id="menu_ciclos_root" name="Ciclos Formativos"/>
<menuitem id="menu_ciclos" name="Ciclos" parent="menu_ciclos_root" action="action_ciclo_formativo"/>
<menuitem id="menu_modulos" name="Módulos" parent="menu_ciclos_root" action="action_modulo"/>
<menuitem id="menu_alumnos" name="Alumnos" parent="menu_ciclos_root" action="action_alumno"/>
<menuitem id="menu_profesores" name="Profesores" parent="menu_ciclos_root" action="action_profesor"/>
```

6.3. Comprobación de Vistas

Con las vistas y los menús de interacción implementados, a continuación realizaremos una comprobación para observar que todo el módulo opera correctamente.

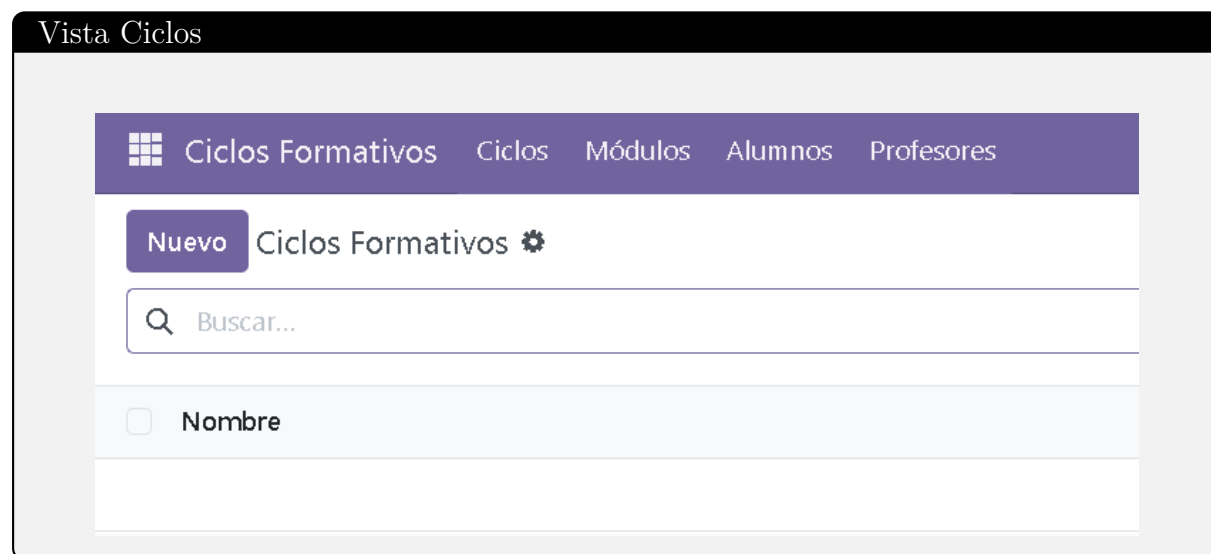
Para ello, buscaremos nuestro módulo y lo instalaremos.





Una vez completada la instalación, ingresaremos al módulo para comprobar su correcto funcionamiento.

Lo primero que realizaremos será la creación de un par de ciclos, así como alumnos y profesores, para luego implementar los módulos con los datos que se necesitan.



Vista Alumnos

 Ciclos Formativos

Ciclos

Módulos

Alumnos

Profesores

Nuevo

Alumnos 



☐	Nombre
☐	Anton
☐	Pedro
☐	Andres
☐	Jose

Vista Profesores

 Ciclos Formativos

Ciclos

Módulos

Alumnos

Profesores

Nuevo

Profesores 



☐	Nombre
☐	Maria
☐	Juana
☐	Andrea

Vista Módulos

Ciclos Formativos

Ciclos

Módulos

Alumnos

Profesores

Nuevo

Módulos

Buscar...

☐	Nombre del módulo	Ciclo formativo	Profesor que imparte
☐	Programación	Inteligencia Artificial	Juana
☐	Base De Datos	Computación	Maria

Vista Módulo con su profesor y alumnos matriculados

Ciclos Formativos

Ciclos

Módulos

Alumnos

Profesores

Nuevo

Módulos

Programación

Programación	Inteligencia Artificial	Juana
Nombre		
Anton		
Jose		
Añadir una línea		

6.4. Configuración de seguridad

Tras confirmar el correcto funcionamiento del módulo, a continuación nos dispondremos a implementar unas configuraciones de seguridad.

Para ello, nos dirigimos a la carpeta **security**, donde crearemos un nuevo archivo llamado **groups.xml** para crear los grupos de seguridad que contendrán los privilegios.

Con el archivo creado, a continuación implementaremos el siguiente código que gestionará los usuarios por grupos.

Roles de los Usuarios

```
<odoo>
  <!-- Grupo Director -->
  <record id="group_director" model="res.groups">
    <field name="name">Director</field>
    <field name="category_id" ref="base.module_category_hidden"/>
  </record>

  <!-- Grupo Profesor -->
  <record id="group_profesor" model="res.groups">
    <field name="name">Profesor</field>
    <field name="category_id" ref="base.module_category_hidden"/>
  </record>
</odoo>
```

Una vez creados los grupos, nos dirigiremos al archivo **ir.model.access.csv**, donde indicaremos los permisos que obtendrá cada uno.

En la actividad se nos indica que los usuarios con el rol de **Director** podrán modificar los registros de los modelos anteriores, mientras que los usuarios con el rol de **Profesor** serán los únicos que podrán ver los datos de los profesores (en modo lectura).

Para ello implementaremos el siguiente código de permisos:

Permisos de acceso

```
id,name,model_id:id,group_id:id,perm_read,perm_write,perm_create,perm_unlink
access_ciclo_formativo_director,ciclo.formativo_director,model_ciclo_formativo,group_director,1,1,1,1
access_ciclo_formativo_profesor,ciclo.formativo_profesor,model_ciclo_formativo,group_profesor,0,0,0,0

access_modulo_director,modulo_director,model_modulo,group_director,1,1,1,1
access_modulo_profesor,modulo_profesor,model_modulo,group_profesor,0,0,0,0

access_alumno_director,alumno_director,model_ciclo_alumno,group_director,1,1,1,1
access_alumno_profesor,alumno_profesor,model_ciclo_alumno,group_profesor,0,0,0,0

access_profesor_director,profesor_director,model_ciclo_profesor,group_director,1,1,1,1
access_profesor_profesor,profesor_profesor,model_ciclo_profesor,group_profesor,1,0,0,0
```

6.5. Comprobación de permisos

Para verificar que la configuración de seguridad funciona correctamente, asignamos los permisos correspondientes a cada usuario.

Permisos de acceso de Director

TECHNICAL

Acceso a la función de exportación ☒

Editor de plantillas de correo ☒

Recibir notificaciones en Odoo ☐

Director ☒

Profesor ☐

Comprobación de Permisos de acceso de Director

Ciclos Formativos Ciclos Módulos Alumnos Profesores

Nuevo

Ciclos Formativos Computación

1 / 1 < >

Computación

Nombre del módulo	Ciclo formativo	Profesor que imparte
Base De Datos	Computación	Maria
Añadir una línea		

Permisos de acceso de Profesor

TECHNICAL

Acceso a la función de exportación ? ☒

Editor de plantillas de correo ? ☒

Recibir notificaciones en Odoo ? ☐

Director ? ☐

Profesor ? ☒

Comprobación de Permisos de acceso de Profesor		
Ciclos Formativos Ciclos Módulos Alumnos Profesores 3 🕒 📅 A		
Ciclos Formativos Computación		1 / 2 < >
Computación		
Nombre del módulo	Ciclo formativo	Profesor que imparte
Base De Datos	Computación	Maria

Como se puede observar:

- El usuario con rol **Director** tiene permisos completos sobre todos los modelos (lectura, escritura, creación y eliminación).
- El usuario con rol **Profesor** solo tiene permiso de lectura sobre el modelo de profesores.

Con esta comprobación final de los permisos de seguridad finalizado, podremos dar por concluido la actividad así como la prelaticia.